

Momkn Pay — Internship Capstone Project Brief

2026-09-20 · @Someone

Duration: 4 weeks · **Tracks:** Backend · iOS (native) · Android (native)

Overview

Interns build **Momkn Pay**, a working bill-payment app for utility services: one backend, one native iOS app, one native Android app, all speaking a single frozen API contract.

The domain mirrors real Momkn work — electricity, water, gas, internet, mobile top-up — without touching production code, NFC hardware or any real payment provider. Every payment is simulated by the backend.

The point of the project is not the feature list. It is making three separate teams agree on a contract, integrate against each other, and ship something that behaves correctly when things go wrong.

Learning outcomes

- Design and consume a versioned REST contract agreed across three teams
- Build an offline-first list with local persistence and client-side search
- Implement a realistic inquiry → confirm → receipt payment flow with idempotency
- Apply AES-256-GCM payload encryption end to end (client encrypts, server decrypts)
- Enforce SSL pinning and store credentials in platform secure storage
- Handle failure paths the way a payment app must: timeouts, expiry, duplicates, declines

Constraints

- 4 calendar weeks, roughly 6 hours per working day
- No real money, no PSP integration, no card schemes, no NFC
- No cross-platform frameworks — iOS is Swift, Android is Kotlin

Sample UI. Every screen in this brief is drawn on a companion design canvas, [Momkn Pay — Capstone Sample UI](#), including the error states and the build tokens both apps share. Interns match it rather than inventing screens. Both this brief and the canvas are private until you share them.

Team structure

One squad of 6–9 interns, split into three tracks, with a rotating integration lead.

Track	Interns	Owns	Blocks everyone if late
Backend	2–3	API, database, mock payment engine, OpenAPI spec, TLS certificate	Yes — both clients wait on it
iOS	2–3	Native Swift app	No
Android	2–3	Native Kotlin app	No

Integration lead — one intern per week, rotating, from any track. Their job for that week: keep the OpenAPI spec and the Postman collection current, run the Wednesday integration session, and log every contract disagreement as an issue. It is a role, not extra credit; it is graded under collaboration.

Contract freeze. The backend track publishes the OpenAPI spec by end of day 2. From day 3 the contract is frozen. Any change after that needs an issue, agreement from both client tracks, and a version bump in the spec's changelog. This rule exists because it is the single most useful thing interns take away from the project.

Backend gets a head start. Backend works alone on days 1–2 while the client tracks build their project skeletons against mock responses from the spec. No client waits idle for an endpoint.

Feature scope

Seven required features. Anything not listed here is out of scope unless it appears under Stretch goals.

F1 — Register

As a new user I create an account with full name, mobile number, email and password.

- Mobile number validated as Egyptian format (01[0125] + 8 digits)
- Password minimum 8 characters, at least one letter and one digit
- Duplicate mobile or email returns a field-level error, not a generic 500
- Server stores only a password hash (bcrypt or Argon2id) — never the password
- On success the user lands on the services list, already signed in

F2 — Login

As a returning user I sign in with mobile number and password and stay signed in across app restarts.

- Returns an access token (15 min) and a refresh token (7 days)
- Tokens stored in Keychain (iOS) / EncryptedSharedPreferences (Android) — never in `UserDefaults`, `SharedPreferences` or a plain file
- A 401 on any call triggers one silent refresh attempt, then a forced logout
- Logout clears tokens, the session key and all cached user data

F3 — Services list, cached locally

As a user I see the catalogue of payable services, and I still see it with no network.

- Fetched once on first login, persisted to the local database (Core Data / SwiftData on iOS, Room on Android)
- App always renders from the local database; network refresh updates the database, the UI reacts to the database
- Pull-to-refresh calls the delta sync endpoint with the stored `syncedAt`
- Grouped by category, with an Arabic and an English name per service
- Airplane-mode test: kill the network, cold-start the app, the list is still there with a stale-data banner

F4 — Search

As a user I filter the services list by typing.

- Search runs against the **local database**, not the network
- Matches Arabic name, English name and category; case- and diacritic-insensitive
- Debounced 300 ms; empty state when nothing matches

F5 — Profile

As a user I view and edit my own details.

- View name, mobile, email, member-since date
- Edit name and email; mobile is read-only
- Change password requires the current password
- Logout with confirmation

F6 — Fees inquiry

As a user I pick a service, enter my subscriber number, and see exactly what I owe before paying anything.

- Input validated client-side against the `inputPattern` the service carries
- Subscriber number is sent **encrypted** (see Security scope)
- Response shows customer name, amount due, service fee, VAT and total, itemised

- Inquiry expires after 5 minutes; a visible countdown, and an expired inquiry cannot be paid
- Handles not-found, already-paid and service-unavailable as distinct UI states

F7 — Payment simulation

As a user I confirm the inquiry with my PIN and get a receipt.

- 4-digit PIN entry, sent encrypted, never logged
- Client generates an `Idempotency-Key` per confirmation; retrying the same key must return the same transaction, not a second one
- Success → receipt screen with transaction ID, timestamp, amounts, copyable reference
- Failure → typed error (insufficient balance, declined, timeout) with a retry path
- Transaction history list, newest first, paginated, each row opening its receipt

Shared API contract

This is the starting contract. Backend publishes it as OpenAPI 3.1 by day 2; after that it changes only through an agreed version bump.

- Base URL: `https://api.momknpay.local/v1` (local) — HTTPS only, no HTTP fallback
- Auth: `Authorization: Bearer <accessToken>` on everything except register, login and refresh
- Every request carries `X-Request-Id` (UUID), `X-Client-Platform` (`ios` | `android`) and `X-Client-Version`
- All money is **integer piastres**, never floats. 4550 = 45.50 EGP
- All timestamps are ISO 8601 UTC

Endpoints

Method	Path	Auth	Purpose
POST	<code>/auth/register</code>	No	Create account
POST	<code>/auth/login</code>	No	Sign in, issue tokens + session key
POST	<code>/auth/refresh</code>	No	Exchange refresh token
POST	<code>/auth/logout</code>	Yes	Revoke refresh token
GET	<code>/profile</code>	Yes	Current user

Method	Path	Auth	Purpose
PATCH	/profile	Yes	Update name, email
POST	/profile/change-password	Yes	Change password
GET	/services	Yes	Full catalogue
GET	/services/sync?since=	Yes	Delta since timestamp
POST	/payments/inquiry	Yes	Fees inquiry (encrypted body)
POST	/payments/confirm	Yes	Pay (encrypted body, idempotent)
GET	/payments/transactions? page=&size=	Yes	History
GET	/payments/transactions/{id}	Yes	Single receipt

POST /auth/login

```
{ "mobile": "01012345678", "password": "Passw0rd" }
```

```
{
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "9f2c...",
  "expiresIn": 900,
  "sessionKey": "base64-32-bytes",
  "user": { "id": "usr_01", "fullName": "Mina Adel", "mobile": "01012345678",
    "email": "m@example.com", "memberSince": "2026-09-20T10:00:00Z" }
}
```

sessionKey is the AES-256 key for this session. It is issued per login, never persisted to disk on the client, and dies with logout.

GET /services

```
{
  "syncedAt": "2026-09-20T10:00:00Z",
  "items": [
    {
      "id": "svc_elec_cairo",
      "nameEn": "Cairo Electricity",
      "nameAr": "كهرباء القاهرة",
      "category": "electricity",
      "iconUrl": "https://cdn.momknpay.local/icons/elec.png",
      "inputLabel": "Subscriber number",
      "inputPattern": "^[0-9]{10}$",
      "minAmount": 500,
      "maxAmount": 500000,
      "isActive": true,
      "updatedAt": "2026-09-18T09:00:00Z"
    }
  ]
}
```

Categories: electricity, water, gas, internet, mobile, landline. Minimum 24 seeded services so search and grouping are worth testing.

/services/sync?since=<ISO> returns the same envelope plus "deletedIds": [], containing only rows changed after since.

POST /payments/inquiry

```
{ "serviceId": "svc_elec_cairo", "payload": "<base64 AES-GCM blob>" }
```

Decrypted payload:

```
{ "subscriberNumber": "1024750891", "nonce": "7f3a...", "ts": 1790000000 }
```

Response:

```
{
  "inquiryId": "inq_8f21",
  "serviceId": "svc_elec_cairo",
  "customerName": "Mina A.",
  "billMonth": "2026-08",
  "amountDue": 24750,
  "serviceFee": 500,
  "vat": 70,
  "total": 25320,
  "currency": "EGP",
  "expiresAt": "2026-09-20T10:05:00Z"
}
```

POST /payments/confirm

Header: `Idempotency-Key: <UUID>` — required.

```
{ "inquiryId": "inq_8f21", "payload": "<base64 AES-GCM blob>" }
```

Decrypted payload: `{ "pin": "1234", "nonce": "...", "ts": 17900000000 }`

```
{
  "transactionId": "txn_5521",
  "status": "SUCCESS",
  "reference": "MP-20260920-5521",
  "paidAt": "2026-09-20T10:02:11Z",
  "total": 25320,
  "service": { "id": "svc_elec_cairo", "nameEn": "Cairo Electricity" }
}
```

Status is `SUCCESS`, `FAILED` or `PENDING`. A repeated `Idempotency-Key` returns the original response with `200`, never a new transaction.

Error envelope

Every non-2xx uses this shape. Clients switch on `code`, never on the message text.

```
{ "error": { "code": "INQUIRY_EXPIRED", "messageEn": "This inquiry has expired.", "messageAr": "انتهت صلاحية الاستعلام.", "field": null } }
```

Code	HTTP	Meaning
<code>VALIDATION_ERROR</code>	400	Bad input; <code>field</code> names the offending key
<code>INVALID_CREDENTIALS</code>	401	Wrong mobile or password

Code	HTTP	Meaning
TOKEN_EXPIRED	401	Access token expired — refresh once
MOBILE_ALREADY_USED	409	Registration conflict
DECRYPTION_FAILED	400	Bad blob, bad key or replayed nonce
SUBSCRIBER_NOT_FOUND	404	No bill for that number
BILL_ALREADY_PAID	409	Nothing due
INQUIRY_EXPIRED	410	Past <code>expiresAt</code>
INSUFFICIENT_BALANCE	402	Simulated decline
SERVICE_UNAVAILABLE	503	Simulated provider outage
RATE_LIMITED	429	Too many attempts

Security scope

Deliberately small: four things, done properly, that interns can explain in a review. This is a teaching exercise, not a production security design — say so to them on day 1.

1. Payload encryption (AES-256-GCM)

Only two fields in the whole app are encrypted: the subscriber number on inquiry and the PIN on confirm. That is the entire scope.

- Algorithm: AES-256-GCM, 12-byte random IV per message, 16-byte auth tag
- Key: the `sessionKey` returned at login, base64 of 32 random bytes, held in memory only
- Wire format: `base64( iv || ciphertext || tag )`
- Plaintext is a JSON object that always carries `nonce` (16 random bytes, hex) and `ts` (unix seconds)
- Server rejects a message whose `ts` is more than 120 seconds old, or whose `nonce` it has already seen, with `DECRYPTION_FAILED`

Platform APIs: `CryptoKit.AES.GCM` on iOS, `javax.crypto.Cipher` with `AES/GCM/NoPadding` on Android, `crypto` (Node) or `javax.crypto` (JVM) on the backend.

Things to tell them explicitly:

- A fresh IV for every single message. Reusing an IV with GCM is a catastrophic break, and it is the mistake they will make.
- Never write your own padding, your own MAC, or ECB mode.
- This sits **on top of** TLS, it does not replace it. Ask each intern why it is still worth doing — the answer is defence in depth and protecting the payload from an intermediary that terminates TLS.

2. SSL pinning

Both clients pin the backend's certificate public key.

- Backend generates a self-signed certificate with `mkcert` or `openssl` on day 1 and publishes its SPKI SHA-256 hash to both client tracks
- iOS: `URLSessionDelegate` implementing `urlSession(_:didReceive:completionHandler:)`, comparing the SPKI hash — not the whole certificate
- Android: `OkHttp CertificatePinner`, plus a Network Security Config for the debug build
- Pin two hashes: the live one and a backup, so rotation is discussed
- **Required demo:** run the app through a proxy (mitmproxy or Charles) with the proxy CA trusted by the device, and show the connection failing. Interns who have not seen pinning actually reject a connection have not learned it.

3. Secure storage

Item	iOS	Android
Access + refresh token	Keychain, <code>kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly</code>	EncryptedSharedPreferences / DataStore + Keystore
Session key	Memory only, cleared on background-to-terminate	Memory only
Cached services	Plain local database — public data, no encryption needed	Same
PIN	Never stored. Anywhere.	Never stored

4. Backend hygiene

- Passwords hashed with `bcrypt` (cost ≥ 12) or `Argon2id` — never MD5, SHA-1, or a home-made hash

- JWT signed HS256 with a secret from an environment variable; no secret in the repository, ever
- Rate limit `/auth/login` and `/payments/confirm` to 5 attempts per minute per user
- Logs must never contain: password, PIN, session key, decrypted payload, full tokens. Log `X-Request-Id` instead and make the interns grep their own logs for these strings as a review item.

Explicitly out of scope

Certificate transparency, App Attest / Play Integrity, jailbreak detection, obfuscation, key rotation protocols, HSM, PCI-DSS compliance. Mention them in the final session as "what a real product adds", and stop there.

Backend track

The backend team owns the contract, the mock payment engine and the certificate. They are the critical path in week 1 and must be told so.

Stack — pick one on day 1 and do not change it:

- Node.js + NestJS + TypeScript + PostgreSQL + Prisma or TypeORM
- Spring Boot + Kotlin/Java + PostgreSQL + JPA
- .NET 8 Web API + PostgreSQL + EF Core

Required

1. Layered structure: controller → service → repository. No database calls inside controllers.
2. DTO validation at the boundary, with field-level error responses matching the error envelope.
3. PostgreSQL with migrations checked into the repository. Tables: `users`, `services`, `inquiries`, `transactions`, `used_nonces`, `refresh_tokens`.
4. Seed script producing ≥ 24 services across all six categories, with Arabic and English names.
5. AES-GCM decrypt helper + nonce replay store (a table or Redis with a 5-minute TTL).
6. Mock payment engine implementing the deterministic rules in the next section.
7. Idempotency: `Idempotency-Key` stored with the transaction; a repeat returns the stored response.
8. JWT issue/verify/refresh with rotation — a used refresh token is revoked.
9. Swagger UI served at `/docs`, generated from the code, kept current.

10. HTTPS locally with the self-signed certificate; publish the SPKI hash to both client teams.
11. `docker-compose up` brings up API + database with one command. This is graded — the clients must be able to run the backend without asking.
12. Postman or Bruno collection covering every endpoint, committed to the repository.

Non-negotiables

- No secret, key or certificate private key committed to git. A `.env.example` instead.
- Integer piastres everywhere. Any float in a money column is a bug.
- Every endpoint returns the agreed error envelope, including unhandled exceptions — a global exception filter, not try/catch scattered around.
- At least 8 unit tests: encryption round-trip, replay rejection, idempotency, inquiry expiry, each mock payment rule.

iOS track

Native Swift, MVVM, no third-party networking library.

Stack

- Swift 5.9+, iOS 16+ deployment target
- SwiftUI (UIKit acceptable if the intern is stronger there — decide per intern on day 1, not mid-project)
- MVVM + a Coordinator or `NavigationStack` path for routing
- `URLSession` with `async/await` — no Alamofire, no Moya
- SwiftData or Core Data for the local cache
- `CryptoKit` for AES-GCM, Keychain via a thin wrapper they write themselves
- Swift Concurrency: `@MainActor` on view models, structured `Task` s

Required

1. A layered project: `Features/` , `Core/Networking/` , `Core/Storage/` , `Core/Security/` , `Core/DesignSystem/` .
2. One generic `APIClient` with a `Endpoint` abstraction, typed errors mapped from the error envelope to a Swift `enum` , and automatic token refresh on 401.
3. `URLSessionDelegate` implementing SPKI pinning.
4. Repository layer returning domain models; view models never see DTOs.
5. Offline-first services list: single source of truth is the local store, network writes into it, UI observes it.

6. `Codable` models with explicit `CodingKeys`, dates decoded with a shared `JSONDecoder` strategy.
7. Loading / empty / error / offline states as first-class UI, not a spinner and a `print`.
8. Keychain wrapper for tokens; session key in memory only.
9. ≥ 8 unit tests: encryption round-trip, error mapping, view model state transitions, search filtering, idempotency key reuse.
10. Accessibility: Dynamic Type supported, all interactive elements labelled.

Non-negotiables

- No force unwraps outside tests. `guard let` or `if let`.
- No network call, no `URLSession`, no database access inside a SwiftUI `View`.
- No secret, key or hash committed as a plain string in source — use an `xconfig` or the build settings, and explain to them why that is still not real security on a client.
- SwiftLint running, warnings at zero before every pull request.

Android track

Native Kotlin, MVVM, mirroring the iOS architecture closely enough that the two teams can review each other's code.

Stack

- Kotlin 1.9+, minSdk 26, targetSdk current
- Jetpack Compose (XML + ViewBinding acceptable per intern, decided on day 1)
- MVVM + `ViewModel` + `StateFlow`, Navigation Compose for routing
- Retrofit + OkHttp + `kotlinx.serialization` or Moshi
- Room for the local cache, Flow-based DAOs
- Hilt for dependency injection
- Coroutines, `viewModelScope`, no `GlobalScope`

Required

1. Multi-module or clearly packaged layers: `feature/`, `core/network/`, `core/database/`, `core/security/`, `core/designsystem/`.
2. Retrofit service interfaces + an OkHttp `Interceptor` chain: auth header, request-id, logging (redacted in release), and an `Authenticator` for token refresh.
3. OkHttp `CertificatePinner` with two SPKI pins, plus a Network Security Config.
4. Repository layer exposing `Flow<Resource<T>>`; UI never touches Retrofit or Room directly.

5. Offline-first list: Room is the single source of truth, network writes into Room, UI collects from Room.
6. `EncryptedSharedPreferences` (or `DataStore` + `Keystore`) for tokens; session key in memory only.
7. AES-GCM via `javax.crypto.Cipher`, IV generated with `SecureRandom` per message.
8. Loading / empty / error / offline as sealed UI state, rendered explicitly.
9. ≥ 8 unit tests: encryption round-trip, error mapping, DAO queries, ViewModel state, search filtering.
10. Configuration-change safety: rotate the device on every screen, nothing is lost or re-fetched.

Non-negotiables

- No `!!` operator outside tests.
- No network or database call from a Composable or an Activity.
- No API call on the main thread; `Dispatchers.IO` for I/O.
- `ktlint` or `detekt` running, clean before every pull request.
- ProGuard/R8 enabled for the release build and the app still works — they should discover this the hard way in week 4, not in production later.

Mock rules

The backend decides outcomes from the **last digit of the subscriber number**.

Deterministic, no randomness, so all three teams can test the same cases and a demo never surprises anyone.

Last digit	Inquiry result	Notes
0	<code>SUBSCRIBER_NOT_FOUND</code> (404)	Empty state on the client
1-5	Success, normal bill	<code>amountDue</code> = digits 3-7 of the number, in piastres
6	Success, but a large bill above <code>maxAmount</code>	Client must block payment with a clear message
7	Success	Confirm then fails with <code>INSUFFICIENT_BALANCE</code>
8	Success	Confirm returns <code>PENDING</code> , resolves to <code>SUCCESS</code> after 10 seconds

Last digit	Inquiry result	Notes
9	BILL_ALREADY_PAID (409)	Distinct UI state, not a generic error

Two more, on the service rather than the number:

- Any service whose id ends in `_slow` — backend sleeps 8 seconds before responding, so clients must show a real loading state and a timeout path
- Any service with `isActive: false` — returns `SERVICE_UNAVAILABLE` (503) and must be visibly disabled in the list, not hidden

Fee formula — same on every environment, so receipts can be checked by hand:

```
serviceFee = \max(500,\; \lceil 0.005 \times amountDue \rceil) \quad \text{vat} =
\lceil 0.14 \times serviceFee \rceil \quad \text{total} = amountDue + serviceFee +
vat
```

Every value in piastres, rounded up to the nearest piastre.

Worked example, the one drawn in the sample UI: subscriber `1024750891` → digits 3–7 are `24750`, so `amountDue` = 24750, `serviceFee` = $\max(500, 124) = 500$, `vat` = 70, `total` = 25320 — 253.20 EGP. Last digit 1, so it takes the happy path.

Seeded test accounts — created by the seed script, same on every machine:

Mobile	Password	PIN	Purpose
01000000001	Passw0rd	1234	Happy path, has transaction history
01000000002	Passw0rd	1234	Fresh account, empty history
01000000003	Passw0rd	9999	Any other PIN is rejected — tests the wrong-PIN path

A wrong PIN returns `VALIDATION_ERROR` with `field: "pin"`. Three wrong PINs in a row on the same inquiry invalidate it.

Week-by-week plan

Four weeks. Each week ends with a working, demonstrable increment — never a week of “still building the architecture”.

Week 1 — Foundations and auth

Track	Work	Done when
Backend	Repository, Docker compose, database + migrations, seed script, register/login/refresh, JWT, password hashing, HTTPS with self-signed cert, OpenAPI published day 2	Both clients can register and log in against a running backend
iOS	Project skeleton, layers, <code>APIClient</code> , error mapping, Keychain wrapper, SSL pinning, register + login screens	Login works end to end, token survives a cold start
Android	Same skeleton, Retrofit + OkHttp + interceptors, CertificatePinner, EncryptedSharedPreferences, register + login screens	Same

Friday demo: register and log in from both devices against one backend, and show the pinning rejection through a proxy.

Week 2 — Catalogue, cache, search, profile

Track	Work	Done when
Backend	<code>/services</code> , <code>/services/sync</code> , profile endpoints, change password, rate limiting, pagination	Endpoints documented and in the Postman collection
iOS	Local store, repository, services list grouped by category, search, profile, logout	Airplane mode: cold start still shows the list
Android	Room, repository with Flow, list, search, profile, logout	Same, plus rotation loses nothing

Friday demo: search working offline on both devices; explain where the data came from.

Week 3 — The payment flow and encryption

Track	Work	Done when
Backend	AES-GCM decrypt + nonce replay store, inquiry with expiry, confirm with idempotency, all mock rules, transactions + receipt endpoints	Every mock rule reachable from the Postman collection
iOS	CryptoKit encryption, inquiry screen, fees breakdown with countdown, PIN entry, receipt, history	The full flow works for digits 1–5 and every error case renders
Android	Same with <code>javax.crypto</code>	Same

Friday demo: a full payment on both devices, then every failure case on purpose. This is the heaviest week — protect it, and do not let week 2 slide into it.

Week 4 — Hardening, tests, polish, demo

Track	Work	Done when
Backend	Unit tests, global exception handling, log audit for secrets, README, final OpenAPI	<code>docker-compose up</code> on a clean machine works first time
iOS	Unit tests, empty/error/offline states, Dynamic Type, SwiftLint clean, README, TestFlight or <code>.ipa</code> build	Release build runs against the backend with pinning on
Android	Unit tests, states, R8 enabled and working, ktlint clean, README, signed APK	Release APK runs with pinning on

Days 18–19: feature freeze. Only bug fixes, tests and documentation after that point.

Day 20: final demo and code review, 90 minutes, each track presenting and each intern answering one security question about their own code.

Reviews and ceremonies

When	What	Length	Who
Daily, 10:00	Standup — yesterday, today, blockers	15 min	Whole squad
Wednesday	Integration session — clients call the real backend together and log mismatches	60 min	Whole squad, integration lead runs it
Thursday	Code review clinic — mentor reviews one pull request per track live, on screen	60 min	Track + mentor
Friday	Demo on real devices, not simulators	45 min	Whole squad + a guest reviewer
Week 2 and 4	1:1 feedback	20 min each	Mentor + intern

Git workflow

- Three repositories: `momknpay-backend`, `momknpay-ios`, `momknpay-android`. A fourth, `momknpay-contract`, holds the OpenAPI spec and the Postman collection, and both client teams watch it.
- `main` is protected. Nobody pushes to it.
- Branches: `feature/<ticket>-<short-name>`, `fix/<ticket>-<short-name>`
- Conventional commits: `feat(auth): add refresh token rotation`
- Every pull request needs one peer approval plus a mentor approval, a description, and screenshots or a recording for UI work
- Pull requests over ~400 changed lines get sent back to be split. Teach this early, it is the habit that sticks.

Issue tracking — Jira, Trello or GitHub Projects. Every feature is a ticket, every ticket has a track label and an acceptance criterion copied from Feature scope. No work without a ticket.

Review gates — a track cannot start the next week's work until the previous week's demo passed. If a track slips, the mentor cuts scope rather than extending the week; interns learn more from shipping a smaller thing on time.

Definition of done

A track is done when every line below is true. Give this list to the interns on day 1 — it is not a surprise at the end.

- ☐ All seven features work against the live backend, on a real device
- ☐ Every error code in the contract renders a distinct, human-readable state
- ☐ The app works offline for the services list and search
- ☐ SSL pinning is on in the release build and demonstrably rejects a proxy
- ☐ Encrypted payloads round-trip; a replayed nonce is rejected
- ☐ No secrets in git history; no PIN, password or key in logs
- ☐ ≥ 8 unit tests passing; linter clean
- ☐ README: how to run, architecture diagram, decisions taken, known limitations
- ☐ Pull requests all reviewed and merged; no work living on a local branch

Final deliverables per track

1. Repository with clean history and a tagged `v1.0` release
2. README with setup steps a stranger can follow
3. Backend: `docker-compose.yml`, OpenAPI spec, Postman collection
4. iOS: signed `.ipa` or TestFlight build
5. Android: signed release APK
6. A 5-minute screen recording of the full happy path plus three failure cases
7. One page of written reflection per intern: what they built, what broke, what they would do differently

Grading rubric

Scored out of 100, per intern, not per track — so a strong intern in a slow team is not punished, and a passenger in a strong team is not carried.

Criterion	Weight	What earns full marks
Functionality	30	All seven features working, every error state handled, nothing crashes in the demo
Architecture and code quality	25	Clean layering, no logic in the UI, readable naming, no dead code, sensible abstractions that are not over-engineered

Criterion	Weight	What earns full marks
Security implementation	15	Pinning proven against a proxy, correct AES-GCM with per-message IV, secure storage, clean logs, and the intern can explain each choice
UX and error handling	12	Loading, empty, error and offline states designed rather than defaulted; Arabic and English names shown; money formatted correctly
Collaboration and process	10	Meaningful commits, reviewed pull requests, tickets kept current, showed up for integration sessions, helped another track
Testing	8	Tests that assert behaviour, not coverage theatre

Passing bar: 60. Below 50 in Security or Architecture is a fail regardless of total, because those are the two things this project exists to teach.

Individual adjustment: the mentor may move any intern ± 10 based on the 1:1s, the reflection and the final demo answers. Interns are told this rule exists on day 1.

A note on the demo. Grade what runs, not what is described. If it is not demonstrable on a real device, it does not score.

Stretch goals

Only after the Definition of done is fully green. An intern who reaches for these with F6 half-finished gets pulled back.

- **Biometric unlock** — Face ID / Touch ID on iOS, BiometricPrompt on Android, gating app re-entry rather than replacing the password
- **Full Arabic localisation with RTL** — including mirrored layouts, Arabic-Indic numeral option, and correctly formatted currency
- **Dark mode** driven by design tokens rather than scattered colour literals
- **Favourites** — pin frequently used services to the top of the list, persisted locally
- **Saved beneficiaries** — store subscriber numbers per service, encrypted at rest
- **Receipt sharing** — render the receipt as a PDF or image and share it
- **Paging** — proper pagination on transaction history (Paging 3 on Android, a cursor-based loader on iOS)
- **CI** — GitHub Actions running lint plus tests on every pull request, for all three repositories

- **Refresh-token rotation with reuse detection** on the backend — if an old refresh token appears, revoke the whole family
- **A shared design system** — one token set, implemented twice, so the two apps genuinely match

Mentor notes

Where interns get stuck, based on how this kind of project usually goes.

Week 1 is the risk. If the backend is late, both client tracks stall and the whole month compresses into three weeks. Mitigation: the contract on day 2 is non-negotiable, and client teams build against stubbed responses generated from the OpenAPI spec until the real backend is up. Consider running a mock server (Prism, WireMock) from the spec on day 2 so clients are never blocked.

Interns will skip the local database. Fetching the list on every screen appearance is easier and looks identical in a demo on office wifi. Catch it by demanding the airplane-mode test at the week 2 demo, on a real device.

IV reuse in AES-GCM. Someone will generate the IV once, store it as a constant, and everything will still work. It is the single most instructive bug in this project — let them find it in code review rather than pointing at it on day 1.

Idempotency gets faked. A client-side boolean that disables the button is not idempotency. Test it by killing the app mid-request and retrying; there must be one transaction, not two.

Pinning is only half-learned. Many interns implement it, see the app work, and conclude it is correct — but the app would also work with no pinning at all. The proxy test is the only proof. Make it a required demo, not a checkbox.

Money as floats. It will appear. Catch it in the first backend review; a receipt showing 45.499999 in the final demo is a memorable but expensive lesson.

Scope creep towards the stretch list. Dark mode and animations are more fun than error states. Hold the line: no stretch work until the Definition of done is green.

The two client teams drift apart. They will make different decisions about the same contract. The Wednesday integration session exists for exactly this — make them run the same test script on both devices side by side.

On the security framing. Say plainly, more than once, that this is a scaled-down teaching exercise, not what Momkn ships. Interns who leave thinking a pinned self-signed certificate plus AES-GCM is a complete payment security model have learned something worse than nothing.